

GIT

GLOBAL INFORMATION TRACKER

- Git is used to track the files.
- It will maintain multiple versions of the same file.
- It is platform independent.
- It is free and open source.
- They can handle larger projects efficiently.
- It is 3rd generation of vcs.
- It is written on c programming
- It came on the year 2005

WORKING DIRECTORY:

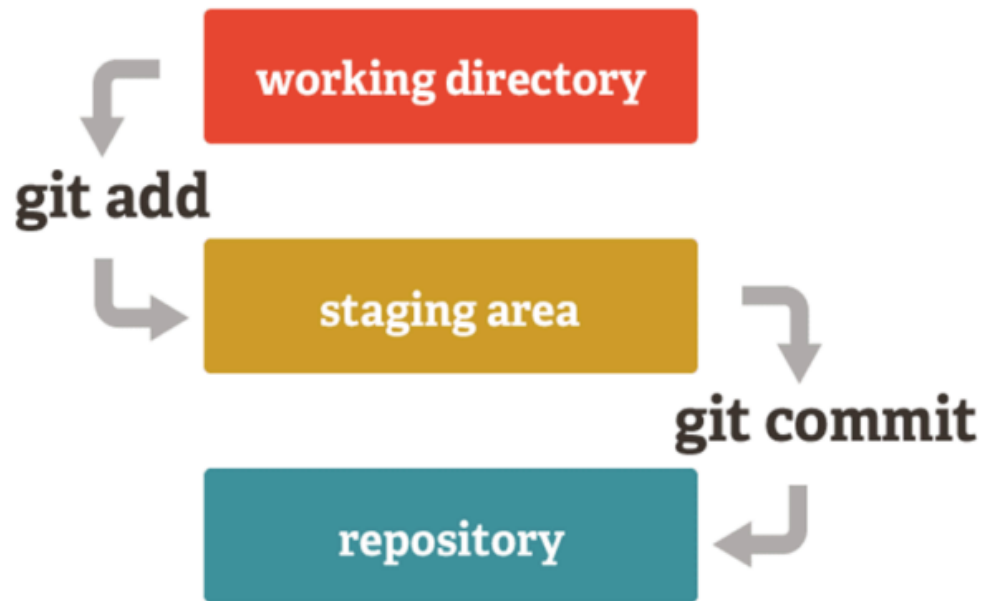
- In this stage git is only aware of having files in the project.
- It will not track these files until we commit those files.

STAGING AREA:

- The staging area is like a rough draft space, it's where you can git add the version of a file or multiple files that you want to save in your next commit.
- In other words, in the next version of your project.

REPOSITORY:

- Repository in Git is considered as your project folder.
- A repository has all the project-related data.
- It contains the collection of the files and also history of changes made to those files.



TYPES OF REPOSITORIES:

LOCAL REPO:

The Local Repository is everything in your .git directory. Mainly what you will see in your Local Repository are all of your checkpoints or commits. It is the area that saves everything (so don't delete it).

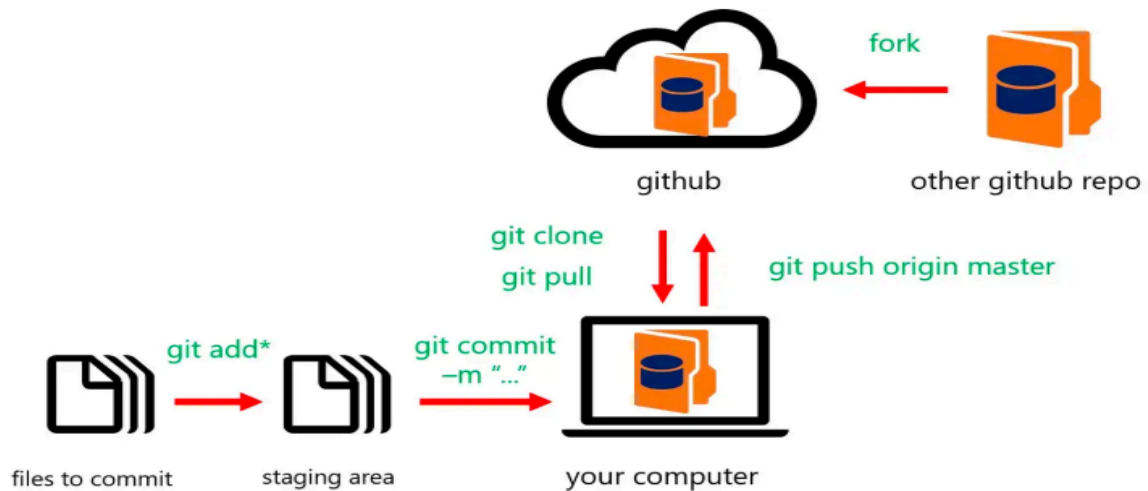
REMOTE REPO:

The remote repository is a Git repository that is stored on some remote computer.

GIT ALTERNATIVES:

- GIT LAB
- SVN
- BIT BUCKET
- P4
- STASH
- HELIX

GIT WORK FLOW:



INSTALL GIT:

`yum install git -y`

yum = Yellowdog Updater Modifier

It is used to update, install or uninstall the packages

To check the version:

1. `git --version`
2. `git -v`
3. `rpm -qa git`

here rpm: redhat package management

-qa: query/verify all packages

To Initialise a git: `git init`.

Here init means initialise.

represents present working directory

when we initialise the git, git folder will gets created which contains some files which relates to git.

GIT ADD:

- Git add command is a straightforward command. It adds files to the staging area.
- We can add single or multiple files at once in the staging area.
- Every time we add or update any file in our project, it is required to forward updates to the staging area.
- The staging and committing are co-related to each other

GIT ADD Commands:

- To track the file: `git add file_name`
- To track the multiple files : `git add aws azure gcp`
- all regular files: `git add *`
- including hidden files: `git add .`

GIT COMMIT:

- It is used to record the changes in the repository.
- It is the next command after the git add.
- Every commit contains the index data and the commit message.

Git Commit Commands:

To commit the file: `git commit -m "commit message" filename`

To commit multiple files: `git commit -m "commit message" file1 file2 file3`

GIT STATUS:

- The git status command is used to display the state of the repository and staging area.
- It allows us to see the tracked, untracked files and changes.
- This command will not show any commit records or information.

GIT Status Commands:

- To check the status: `git status`

GIT CONFIGURE:

if you want to give your username and E-mail id to those commits then

- `git config user.name "username"` : command to set username
- `git config user.email "userxyz@gmail.com"` : command to set mail-id

GIT LOG:

Git Log is a command in the Git version control system that shows a history of commits made in a Git repository. It displays a list of past changes, including information like the commit message, author, date, and a unique identifier for each commit.

COMMANDS:

<code>git log</code>	Used to see the history of the git
<code>git log --oneline</code>	Used to see only commit ID's and messages
<code>git log --pretty=oneline</code>	Used to get only commit ID's and messages
<code>git log -1</code>	Used to see the latest commit
<code>git log -3</code>	Used to see latest 3 commits
<code>git log --follow --all filename</code>	Used to see the no of commits for a single file
<code>git log --graph --oneline --all</code>	See all the history in graph

GIT AMEND:

It is a Git command used to make changes to the most recent Git commit. It allows you to edit the commit message, add additional changes, or both.

`git commit --amend -m "message"` Used to change the commit message for a latest commit
`git commit --amend --author "username "` Used to change the author of latest commit
`git commit --amend --no-edit` Used to commit the changes with previous commit

GIT RESET:

Git Reset is a command in Git that allows you to move the HEAD and the current branch pointer to a specific commit, effectively "rewinding" or "resetting" your project's state. It's commonly used to undo changes or to unstage commits.

COMMANDS:

<code>git reset --hard HEAD~1</code>	Used to delete the latest commit along with the changes git
<code>reset --hard HEAD~3</code>	Used to delete the latest 3 commits along with the changes
<code>git reset --soft HEAD~1</code>	Used to delete only commits but not actions/changes git
<code>reset --soft HEAD~3</code>	Used to delete only latest commits but not actions/changes

GIT REVERT:

Git Revert is a command that allows you to undo or reverse the changes made in a previous commit. It creates a new commit that undoes the changes introduced by the specified commit, effectively taking your code back to a previous state without deleting commit history.

COMMAND:

`git revert commit_id` Used to delete a particular commit action and add a new commit for the change

GIT IGNORE:

- It will be useful when you don't want to track some specific files then we use a file called .gitignore
- create some text files and create a directory with "jpg" files.

GIT BRANCHES:

- A branch represents an independent line of development.
- The git branch command lets you create, list, rename, and delete branches.
- The default branch name in Git is master.
- allows you to work on different features or changes to your code independently, without affecting the main or other branches.
- It's a way to organize and manage your code changes, making it easier to collaborate and maintain your project.

COMMANDS:

<code>git branch</code>	Used to see the list of branches
<code>git branch branch-name</code>	To create a branch
<code>git checkout branch-name</code>	To switch one branch to another
<code>git checkout -b branch-name</code> a time	Used to create and switch a branch at
<code>git branch -m old-branch new-branch</code>	Used to rename a branch
<code>git branch -d branch-name</code>	To delete a branch
<code>git branch branch-name deleted-branch-id</code>	Used to get deleted branch id
<code>git branch -D branch-name</code>	To delete a branch forcefully

The -d option will delete the branch only if it has already been pushed and merged with the remote branch. Use -D instead if you want to force the branch to be deleted, even if it hasn't been pushed or merged yet. The branch is now deleted locally. Now all the things you have done is on your local system.

GIT MERGE:

Git merge is a command used in the Git version control system to combine changes from one branch.

To merge: `git merge branch_name`

GIT CHERRY-PICK:

Git cherry-pick is a command in Git that allows you to take a specific commit from one branch and apply it to another branch. It's like picking a cherry (commit) from one branch and adding it to another branch, allowing you to selectively copy individual commits without merging the entire branch.

Command: `git cherry-pick commit_id`

GIT RESTORE:

Git restore command is used to restore the deleted data/file

Command: `git restore filename`

GIT MERGE CONFLICTS:

GIT makes merging super easy!

CONFLICTS generally arise when two people have changed the same lines in a file (or) if one developer deleted a file while another developer is working on the same file!

In this situation git cannot determine what is correct!

Let's understand in a simple way!

```
cat>file1 : hai all
```

```
add & commit
```

```
git checkout -b branch1
```

```
cat>file1 : 1234
```

```
add & commit
```

```
git checkout master
```

```
cat>>file1 : abcd
```

add & commit

```
git merge branch1 : remove it
```

```
git diff file1
```

```
vim file1
```

```
git add
```

```
git commit -m "final commits"
```

NOTE: Don't give file name on commit

Identify Merge Conflicts:

see the file in master branch then you will see both the data in a single file including branch names that are dividing with conflict messages

Resolve Conflicts:

open file in VIM EDITOR and delete all the conflict dividers and save it!

add git to that file and commit it with the command (git commit -m "merged and resolved the conflict issue in abc.txt")

GIT REBASE:

Git rebase is a Git command used to incorporate changes from one branch into another. It allows you to re-organize and streamline the commit history by moving or combining commits from one branch onto another.

Command: `git rebase-branch`

GIT STASH:

Using the git stash command, developers can temporarily save changes made in the working directory. It allows them to quickly switch contexts when they are not quite ready to commit changes. And it allows them to more easily switch between branches. Generally, the stash's meaning is "store something safely in a hidden place."

COMMANDS:

<code>git stash</code>	To delete the changes permanently
<code>git stash save "message"</code>	To save the stash along with the message
<code>git stash apply</code>	To get back the data again
<code>git stash list</code>	To get the list of stashes
<code>git stash clear</code>	To clear all stashes
<code>git stash pop</code>	To delete the first stash
<code>git stash drop</code>	Used to delete the latest stash
<code>git stash drop stash@{2}</code>	Used to delete a particular stash

GIT TAGS:

Git tags are markers or labels that you can place on specific commits (versions) in a Git repository. These tags provide a way to give meaningful names to important points in the project's history, such as software releases or significant milestones.

COMMANDS:

<code>git tag</code>	To see the list of tags
<code>git tag tagname</code>	To create normal tag
<code>git tag -a -m "message" tagname</code>	To create annotated tag
<code>git tag -d tagname</code>	To delete a tag
<code>git show tagname</code>	To see the details of the tag

GIT-HUB

- GitHub is a web-based platform used for version control.
- It simplifies the process of working with other people and makes it easy to collaborate on projects.
- Team members can work on files and easily merge their changes in with the master branch of the project.

COMMANDS:

<code>git remote add origin repo-url</code>	link local-repo to central-repo
<code>git remove -v</code>	used to get the linked repo in github
<code>git push -u origin branch-name</code>	push the code from local to central
<code>git push -u origin branch-1 branch-2</code>	used to push the code to multiple branches
<code>git push -u origin --all</code> time	used to push the code to all branches at a time
<code>git clone repo-url</code>	used to get the code from central to local
<code>git pull origin branch</code>	used to get the changes from central to local
<code>git fetch branch-name</code>	used to fetch the data from central to local
<code>git fetch --all</code> in github	used to fetch the changes from all branches
<code>git merge origin/branch</code> local	used to merge the changes from central to local
<code>git push -u origin --delete branch-name</code>	used to delete the github branch from local
<code>git remote rm origin</code>	used to unlink the github-repo
<code>git remote rename old -link new-link</code>	used to change the repo